

The documentation for pkm (InterGenOS Package Manager) shows a mature design that balances the simplicity of Slackware-style manifests with the modern reliability of a SQLite backend. The natural-language CLI interface is a great touch for a custom distribution.

Here is a technical assessment based on the specific areas you highlighted:

1. SQL Injection Assessment

The use of db.py generally follows best practices by using parameterized queries (e.g., `execute(sql, params)`).

- **Safety:** By using the standard library's `sqlite3` parameter markers (`?`), you are effectively protected against basic SQL injection.
- **Observation:** Ensure that any dynamic table names or column names (if used in future pkm search filters) are strictly allow-listed, as parameters cannot be used for identifiers, only for values.

2. Archive Extraction & "Tar Slip" Security

Your concern regarding **Tar Slip (Path Traversal)** is critical.

- **The Risk:** A malicious or malformed `.igos.tar.gz` could contain a file named `../..../etc/shadow`.
- **The Solution:** Within `installer.py`, before calling the extraction, you should verify that the absolute path of every member in the archive starts with the intended target directory.
- **Symlink Attacks:** During extraction, if a package contains a symlink to `/etc` and a subsequent file in the *same* archive is written to that symlink path, it can overwrite host files.
- **Recommendation:** Use `tarfile.extractall()` with the `filter='data'` (available in Python 3.12+) which is designed to mitigate these specific filesystem-traversal risks.

3. Config File Preservation

The logic for skipping `/etc` files is well-conceived.

- **Implementation:** By comparing the current file hash on disk against the stored hash in the SQLite database, you correctly identify "user-tainted" configs.
- **Edge Case:** If a user changes a file's permissions or ownership but not its content, the SHA256 hash remains the same. You may want to decide if "metadata-only" changes should also trigger preservation.

- **Improvement:** Rather than just skipping the removal, consider the `pkm-new / pkm-old` pattern used by Arch or Slackware, where the manager notifies the user that a conflict exists between the package default and the user's version.

4. Dependency Resolution (Removal Safety)

The reverse-dependency check in `resolver.py` is the most important "guardrail" for system stability.

- **Logic:** The check for if name in `self.db.get_all_dependencies()` before removal is correct.
- **The "Cascade" Risk:** Ensure that your resolver distinguishes between *required* dependencies and *optional* ones (if your YAML templates support them).
- **Circular Block:** If two packages depend on each other, the current logic might prevent the removal of either. You may need a `--recursive` or `--force` flag for such scenarios, with a heavy warning to the user.

5. File Operation Safety & TOCTOU

- **The Race Condition:** Time-of-check to time-of-use (TOCTOU) is a risk during the "Verify then Install" phase. If a symlink is replaced between the time `pkm` checks a directory and the time it writes to it, it could lead to unauthorized writes.
- **Mitigation:** When deploying files, using `os.rename()` for atomic replacement of files is safer than deleting and then writing.
- **Root Support:** The `--root` implementation is clean. However, ensure that all paths within the database remain relative to the root so that the database is portable if the drive is moved to another mount point.

Summary of Recommendations

Category	Finding	Recommendation
Integrity	SHA256 is only checked on install.	Add a <code>pkm verify</code> command that re-runs hashes on all installed files to detect "bit rot" or unauthorized changes.

Performance	SQLite is fast, but individual inserts are slow.	Use a single transaction (BEGIN/COMMIT) for the entire installation process to ensure atomicity and massive speed gains.
UX	Natural language is great but can be ambiguous.	Ensure pkm install bash supports exact matches first, then fuzzy matches, to prevent accidental installations of similar-sounding packages.
Safety	File removal logic.	When removing directories, ensure they are actually empty. Your current rmdir logic seems to handle this, but adding a check to ensure you never rmdir / or /usr is a vital safety switch.

Overall, pkm provides a solid foundation for InterGenOS. The combination of a relational database for metadata and flat manifests for file tracking provides the "best of both worlds" for a source-based distribution.